Model Selection for Credit Card Fraud Detection:

A Comparative Study of Decision Trees, k-NN, and Naïve Bayes on Imbalanced Data

WONG Chi Kin

Table of contents

- 1 Introduction: Problem Definition
- 2 Data Collection, Understanding, and Exploratory Data Analysis
  - 2.1 Data Source and Structure Overview
  - 2.2 Target Variable Analysis
  - 2.3 Exploratory Feature Examination (EDA)
- 3 Data Preprocessing and Feature Engineering
  - 3.1 Basic Data Cleaning
  - 3.2 Stratified Sampling Strategy
  - 3.3 Temporal Parsing and Behavioral Feature Engineering
  - 3.4 Geospatial Distance and High-Cardinality Reduction
  - 3.5 Removal of Irrelevant Attributes
  - 3.6 Preprocessing Recipe and Standardization
  - 3.7 Handling Class Imbalance (SMOTE) and Final Output
- 4 Methodology and Results: Modeling training and Validation
  - 4.1 Overview of Modeling Strategy
  - 4.2 Decision Tree (rpart)
    - 4.2.1 Model Summary and Parameter Tuning
    - 4.2.2 Feature Importance (Top 10)
    - 4.2.3 Model Performance – Validation Set
  - 4.3 k-Nearest Neighbors (kNN)
    - 4.3.1 Model Summary and Parameter Tuning
    - 4.3.2 Model Performance – Validation Set
  - 4.4 Naïve Bayes
    - 4.4.1 Model Summary and Parameter Tuning
    - 4.4.2 Model Performance – Validation Set
  - 4.5 Model Comparison and Selection
- 5 Final Testing, Discussion, and Conclusion
  - 5.1 Final Test on the Selected Decision Tree Model
  - 5.2 Discussion: Performance Comparison and Business Implications
    - 5.2.1 Analysis of Performance Variance (Validation vs. Test)
    - 5.2.2 Benchmarking Against Industry Standards
  - 5.3 Conclusion
- References
- Appendix
  - Figures (A.1–A.7)

1 Introduction: Problem definition

With the rapid growth of e-commerce and digital payments, credit card fraud has become a key challenge for financial institutions. It causes direct losses, increases operational risk, and erodes customer trust—core concerns in enterprise risk analytics. Traditional rule-based systems are static and reactive, making them ineffective against evolving fraud patterns.

From a risk analytics perspective, transaction-level fraud detection exemplifies predictive analytics within credit and operational risk. Machine learning enables institutions to quantify risks, prioritize reviews, and improve controls.

This project is designed to identify the optimal machine learning model for credit card fraud detection by rigorously comparing Decision Tree, k-Nearest Neighbors (kNN), and Naïve Bayes on a highly imbalanced dataset (fraud rate < 0.6%). Model performance is evaluated using standard metrics (AUC, Recall, Precision, F1-Score) (Saito & Rehmsmeier, 2015). For a model to be deemed successful, it must clear two critical hurdles: first, deliver superior performance in a comparative validation; and second, prove its practical worth by achieving industry benchmarks (AUC ≥ 0.85, Recall ≥ 0.75) on a pristine, held-out test set (Dal Pozzolo et al., 2014).

We focus on these three classic baselines because they embody distinct learning logics—rule-based splitting (Decision Tree), distance-based local voting (kNN), and probabilistic inference (Naïve Bayes). This diversity allows a clean comparison of how different inductive biases behave under severe class imbalance, which is central to fraud detection. Lightweight and interpretable baselines also facilitate rapid iteration on large transactional data and

straightforward benchmarking against industry metrics and prior studies. Detailed model mechanisms and tuning are presented in the Methodology section.

The workflow includes:

(1) Introduction- Problem definition

(2) Data Collection, Understanding and Exploratory Data Analysis

(3) Data preprocessing and Feature engineering

(4) Methodology and Results: Modeling training and Validation

(5) Final testing, Discussion and Conclusion

The next section presents the data characteristics and exploratory findings that form the foundation for subsequent modeling.

---

2 Data Collection, Understanding and Exploratory Data Analysis

The data used in this project were obtained from Kaggle (Kartik, 2020) and consist of a training dataset (fraudTrain.csv) and a test dataset (fraudTest.csv).

The training set contains 1,296,675 records and 23 variables (11 numerical, 10 categorical). Key features include transaction amount, time, merchant location, and demographics. Non-predictive identifiers (e.g., trans_num, first, dob) were removed to avoid data leakage (Kuhn & Johnson, 2013). Data types and structure were verified using str(), summary(), and tidyverse functions.

2.2 Target Variable Analysis

The target variable is_fraud is binary. The original training and test sets exhibit severe class imbalance, with fraud rates of 0.5789% (approx. 1:172) and 0.386% respectively—a characteristic of financial fraud datasets (Dal Pozzolo et al., 2014). Given that accuracy is a misleading metric under such imbalance, model performance will be assessed using AUC, Recall, Precision, and F1-Score. To enhance the model's ability to recognize fraud, the Synthetic Minority Over-sampling Technique (SMOTE) will be applied to the training data (Chawla et al., 2002).

2.3 Exploratory Feature Examination (EDA)

(1) Transaction amount amt is right-skewed (mean = 70.4, median = 47.5); frauds show a much higher median (396), so log transformation is applied.

(2) Transaction time trans_date_trans_time shows peak activity at 18:00–23:00, with frauds clustering at 22:00–01:00. Temporal features (hour, is_weekend, is_night) were derived.

(3) City population city_pop ranges from 23 to 2.9M; log transformation reduces dispersion, indirectly helping mitigate the risk of overfitting in subsequent model analysis.

(4) Haversine distance between customer and merchant was computed to capture spatial risk.

Overall, the dataset is both extensive and extremely imbalanced. Transaction amount and timing are key fraud indicators, while spatial and demographic features provide additional context.

---

3 Data Preprocessing and Feature Engineering

3.1 Basic Data Cleaning

Column names were standardized, duplicates removed, and the target variable converted to a factor ("no", "yes") to prevent class misrecognition during modeling (Kuhn & Silge, 2022). Final training and test sets comprise 1,296,675 and 555,719 records, respectively.

3.2 Stratified Sampling Strategy

Training Set Sampling: A stratified sample of 20,000 records was drawn from the original ~1.3M training set using is_fraud as the key. This preserved the class distribution while enhancing computational efficiency.

Test Set Sampling: Although retaining the full test set is ideal (Kuhn & Silge, 2022), computational limits necessitated a stratified sample of 50,000 records. The sampled fraud rate (0.39%) closely matches the original (0.386%), confirming the sample's statistical representativeness and ensuring valid performance evaluation (Kuhn & Silge, 2022).

3.3 Temporal Parsing and Behavioral Feature Engineering

Using lubridate, timestamps were parsed into hour, weekday, is_weekend, and time_segment. For each card, transactions were chronologically sorted to compute:
(1) time since previous transaction, and (2) total count and amount within 1h and 24h—features commonly used to detect abnormal behavior (Carcillo et al., 2019).

3.4 Geospatial Distance and High-Cardinality Reduction

Haversine distance (dist_cust_merc) was calculated to capture spatial risk (Sinnott, 1984; Bolton & Hand, 2002). High-cardinality features like merchant category and occupation were reduced to top 100 and 30 levels, with others grouped as "Other". amt was log-transformed and standardized (Kuhn & Johnson, 2013) for model stability.

3.5 Removal of Irrelevant Attributes

To mitigate overfitting risks, identifiers such as transaction numbers and customer names were removed to prevent overfitting and ensure models learn general patterns (Torgo, 2010).

3.6 Preprocessing Recipe and Standardization

A preprocessing pipeline was built using the recipes package in the tidymodels framework. All transformations were based on the training set and then applied to the test set. Steps included merging rare categories, handling new levels, imputing missing values, one-hot encoding, removing zero-variance features, and Z-score standardization. This ensured consistent processing and reproducible model evaluation (Kuhn & Silge, 2022).

3.7 Handling Class Imbalance (SMOTE) and Final output

To prevent the model from focusing disproportionately on the majority (non-fraud) class, the Synthetic Minority Over-sampling Technique (SMOTE) was applied after feature transformation of the training data (Chawla et al., 2002). Initial testing employed a 25% synthetic oversampling ratio, but this approach demonstrated overfitting characteristics in validation. The final configuration targeted a 5 % fraud rate (actual 4.82 %), achieving a balance between generalization and minority-class detection performance (Blagus & Lusa, 2013). The test set was kept untouched to maintain realistic performance evaluation.

Final output comprised two standardized datasets for modeling: train_stratsample_smote and test_stratsample_50k_fe.csv, ensuring comparable feature structures (Kuhn & Silge, 2022).

---

4. Methodology and Results: Modeling training and Validation

4.1 Overview of Modeling Strategy

Three classification models were trained and validated on the same imbalanced dataset using a consistent preprocessing pipeline: Decision Tree(rpart), k-Nearest Neighbors (kNN) and Naïve Bayes

All models used an 80% stratified training set with SMOTE oversampling (~5% fraud rate), and were evaluated on a 20% holdout set (n = 4,177). Performance was measured using AUC, Recall, Precision, and F1 Score.

4.2.1 Model Summary & Parameter Tuning - Decision Tree

Decision Trees recursively partition the feature space by selecting splits that maximize class purity (e.g., Gini impurity reduction), yielding an interpretable set of if–then rules; model capacity is controlled by tree depth and post-pruning via a complexity parameter that penalizes overly complex trees (Breiman et al., 1984).

In this study, the tree was trained on 16,714 records, with 10-fold cross-validation selecting a complexity parameter (cp) of 0.0010. The final tree reached a depth of 10 with 67 nodes. Tuning revealed that this cp value yielded the best cross-validation AUC (0.9928), balancing complexity and predictive power (Appendix 1, Figure A.1). As a rule-based model, it splits data on key features (e.g., amount, merchant type), offering high interpretability suitable for fraud detection (Bahnsen et al., 2016). The tree structure is shown in Appendix 1, Figure A.2

4.2.2 Feature Importance (Top 10) - Decision Tree

Feature importance, based on Gini impurity reduction, identified four key categories (Appendix 1, Figure A.3):

1. Transaction Amount

   amt, log_amt, amt_sum_1h, amt_sum_24h

   → Transaction value and recent spending are the strongest predictors of fraud.

2. Transaction Time

   hour, daypart_late_night, daypart_evening

   → Fraud is more likely to occur during off-peak hours.

3. Transaction Type

   category_gas_transport, category_grocery_pos

   → Certain merchant categories are more commonly associated with fraud.

4. Transaction Frequency

   time_since_last

   → Short intervals between transactions may indicate suspicious behavior.

4.2.3 Model Performance - Decision Tree (Validation Set)

The decision tree model was evaluated using a hold-out validation set of 4,177 transactions. The confusion matrix summarizes the prediction outcomes:

Confusion Matrix- Decision Tree		
Predicted Yes	18(FP)	198(TP)
Predicted No	3958(TN)	3(FN)
	Actual No	Actual Yes

In practical terms:

True Negatives (TN): 3,958 – Correct non-fraud

The model correctly accepted legitimate transactions. This is the most common and correct outcome.

False Positives (FP): 18 – Wrong catch

Legitimate transactions were mistakenly flagged as fraud (false alarms), potentially causing customer inconvenience.

False Negatives (FN): 3 – Missed catch

Fraudulent transactions were incorrectly classified as legitimate. This is the most critical error, leading to possible financial loss.

True Positives (TP): 198 – Correct catch

The model successfully detected fraudulent transactions, fulfilling its core detection objective.

These four outcomes enable the calculation of standard classification metrics that quantitatively capture the model's performance characteristics:

- Accuracy: Overall prediction correctness = (TP + TN) / (TP + TN + FP + FN)
- Recall: Fraud case detection rate = TP / (TP + FN)
- Precision: Fraud alert accuracy = TP / (TP + FP)
- F1 Score: Precision-recall balance = 2 × (Precision × Recall) / (Precision + Recall)

These metrics provide a comprehensive evaluation for imbalanced tasks like fraud detection.

Classification metrics- Decision Tree				
Recall	Precision	F1_Score	Accuracy	Balanced Accuracy
0.9851	0.9167	0.9496	0.9950	0.9903

The model detected 198 of 201 fraud cases (Recall: 0.9851) with only 18 false alarms (Precision: 0.9167). The F1 Score of 0.9496 reflects excellent balance between sensitivity and precision. With a Balanced Accuracy of 0.9903, incorporating both high recall and specificity (99.55%), the decision tree proves highly effective for fraud detection—particularly in minimizing false negatives, which is crucial in financial risk management.

4.3.1 Model Summary & Parameter Tuning- k-Nearest Neighbors

k-Nearest Neighbors classifies a new sample by majority vote among its k closest training points under a chosen distance metric (Euclidean distance in this study), making it a non-parametric, locality-based method (Cover & Hart, 1967). Smaller k captures finer local structure but increases variance; larger k smooths decision boundaries but may miss minority patterns. All features were standardized prior to distance computation. Through 10-fold cross-validation, k = 5 yielded the highest AUC (0.996). Performance declined for larger k, suggesting that smaller neighborhoods better capture local fraud patterns in our feature space (Appendix 1, Figure A.4).

4.3.2 Model Performance – kNN (Validation Set)

Confusion Matrix-kNN		
Predicted Yes	77(FP)	199(TP)
Predicted No	3899(TN)	2(FN)
	Actual No	Actual Yes

Classification metrics-kNN				
Recall	Precision	F1_Score	Accuracy	Balanced Accuracy
0.9900	0.7210	0.8344	0.9811	0.9853

The kNN model achieved very high recall 0.9900, correctly detecting 199 out of 201 fraud cases. The model produced 77 false positives, leading to a precision score of 0.7210. This outcome highlights a key characteristic: while the model demonstrates strong capability in detecting potential fraud, it also exhibits a tendency to misclassify legitimate transactions as fraudulent. The resulting F1 score of 0.8344 effectively captures this dynamic, reflecting a design that prioritizes sensitivity over precision in fraud identification.

Despite the high overall accuracy (0.9811) and AUC (0.9960), the reduced precision could lead to a higher number of false alarms in practical applications, which may affect operational efficiency and user trust.

4.4.1 Model Summary & Parameter Tuning– Naïve Bayes

The Naïve Bayes classifier predicts class probabilities under a conditional independence assumption between features. While this assumption is simplistic, the method can be effective for high-dimensional, imbalanced data common in financial fraud detection (Bahnsen et al., 2016). A 10-fold CV tuning process evaluated Laplace smoothing parameters (0–5) with and without kernel density estimation. The optimal configuration (laplace = 0, usekernel = TRUE) achieved the highest cross-validation AUC (0.9387), showing that kernel estimation boosted performance while Laplace smoothing offered no improvement (Appendix 1, Figure A.5). Despite the strong AUC, using the default 0.5 decision threshold in our validation yielded no fraud detections (recall = 0), indicating that threshold selection—rather than ranking ability—was the limiting factor in this setting.

4.4.2 Model Performance – Naïve Bayes (Validation Set)

Confusion Matrix- Naïve Bayes		
Predicted Yes	0(FP)	0(TP)
Predicted No	3976(TN)	201(FN)
	Actual No	Actual Yes

Classification metrics- Naïve Bayes				
Recall	Precision	F1_Score	Accuracy	Balanced Accuracy
0.0000	N/A	N/A	0.9519	0.5000

The Naïve Bayes model demonstrated a complete failure in fraud detection, achieving a recall of 0.0000 by incorrectly classifying all 201 fraudulent transactions. This misclassification resulted in 201 false negatives, rendering both precision and F1 score incalculable. Mechanistically, Naïve Bayes computes posterior probabilities from priors and likelihoods; under extreme class imbalance (even with SMOTE raising the minority share to about 5%) and substantial feature overlap with non-fraud, the posterior for fraud tends to stay below 0.5—so at the default threshold the model flags none (recall≈0). While the model showed some ranking capability with an AUC of 0.9665, its practical performance at the default classification threshold proved entirely ineffective for real-world fraud detection. This significant discrepancy between probabilistic ranking and actual classification output underscores the model's limitations in handling highly imbalanced financial data (Carcillo et al., 2019).

4.5 Model Comparison and Selection

The comparative performance of the three models is summarized in the table below and visually supported by the ROC curves in Appendix 1, Figure A.6.

	Model	AUC	Recall	Precision	F1_Score	Accuracy	Specificity	Balanced_Accuracy
1	Decision Tree	0.9995	0.9851	0.9167	0.9496	0.9950	0.9955	0.9903
2	kNN	0.9960	0.9900	0.7210	0.8344	0.9811	0.9806	0.9853
3	Naive Bayes	0.9665	0.0000	NA	NA	0.9519	1.0000	0.5000

In credit card fraud detection, the cost of a false negative significantly outweighs that of a false positive, making Recall a more critical metric than AUC for model evaluation (Dal Pozzolo et al., 2014; Carcillo et al., 2019). Accordingly, the Naïve Bayes model is disqualified, as it completely failed to detect fraud, yielding a Recall of 0.0000 despite a moderate AUC.

Between the remaining models, kNN achieved a marginally higher Recall (0.9900) than Decision Tree (0.9851), but with substantially lower Precision (0.7210 vs. 0.9167). This higher false positive rate would increase operational costs and impair customer experience (Bahnsen et al., 2016). The Decision Tree's superior F1-Score (0.9496) versus kNN (0.8344) demonstrates a better balance between detecting fraud and minimizing false alarms.

Therefore, the Decision Tree is selected for final testing, offering the most balanced performance and practical interpretability for deployment.

---

5. Final testing, Discussion and Conclusion

5.1 Final Test on the Selected Decision Tree Model

The final Decision Tree model was evaluated on an independent, untouched test set (test_stratsample_50k_fe.csv) to assess its real-world generalization capability. This set preserved the original, highly imbalanced data distribution (fraud rate: 0.39%), providing a realistic performance estimate for production deployment.

The confusion matrix and key metrics from this final evaluation are summarized below:

Confusion Matrix- Decision Tree (Test set)		
Predicted Yes	348(FP)	146(TP)
Predicted No	49458(TN)	47(FN)
	Actual No	Actual Yes

The confusion matrix revealed the expected trade-offs in a real-world scenario, with 47 false negatives (missed frauds) and 348 false positives (false alarms).

More importantly, the quantitative metrics derived from these results highlight a significant performance shift from the validation phase, as detailed in the following comparison:

Metric	Validation Set (SMOTE) (from part 4.3.2 result)	Test Set (No SMOTE)
AUC	0.9995	0.8892
Recall	0.9851	0.7565
Precision	0.9167	0.2955
F1-Score	0.9496	0.4250
Balanced Accuracy	0.9903	0.8747

(The comparative ROC curves (Validation vs. Test Set) are provided in Appendix 1, Figure A.7)

5.2 Discussion: Performance Comparison and Business Implications

5.2.1 Analysis of Performance Variance

A noticeable performance difference exists between the validation set (Recall: 0.9851) and the test set (Recall: 0.7565). This is not an error but an expected and insightful finding. The disparity

arises because the validation set was derived from SMOTE-augmented data, creating an "idealized" classification environment, while the test set reflects the "harsh reality" of intrinsic class imbalance. This variance highlights a fundamental challenge in moving from laboratory experiments to production systems (López et al., 2013).

Crucially, this report provides a realistic rather than overly optimistic performance estimate, which is essential for stakeholder decision-making (Bahnsen et al., 2016).

5.2.2 Benchmarking Against Industry Standards

In operational fraud detection, a model is typically considered effective if it achieves AUC ≥ 0.85 and Recall ≥ 0.75, balancing discrimination ability with sufficient fraud coverage (Dal Pozzolo et al., 2014). Our final model meets both industry benchmarks (AUC=0.889, Recall=0.757), confirming its practical utility and readiness for deployment.

The performance gap, while expected, suggests that even the final 5% SMOTE configuration may introduce some degree of overfitting. Future iterations could explore more conservative oversampling ratios (e.g., 3%) to better align with the natural class imbalance. Additionally, applying dimensionality reduction during preprocessing—such as Principal Component Analysis (PCA)—can further mitigate overfitting by retaining only the most informative components, thereby reducing model variance and noise (Jolliffe & Cadima, 2016).

5.3 Conclusion

This project systematically compared three machine learning models for credit card fraud detection. The Decision Tree was identified as the optimal model due to its balanced performance and interpretability. Although Naïve Bayes achieved a relatively high AUC (0.9665), it failed to detect any fraudulent transactions, yielding a recall of 0.0000. This

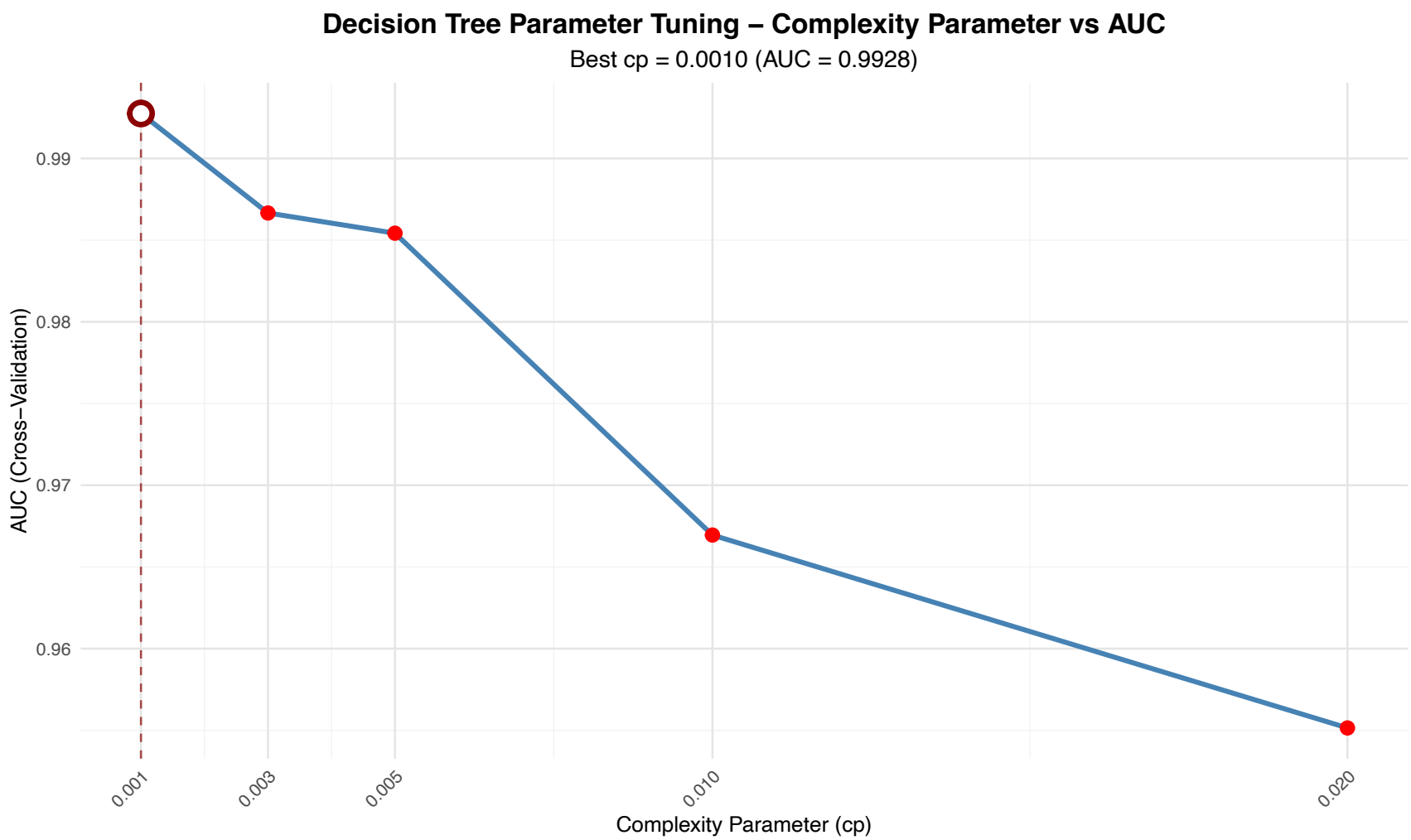
highlights a critical limitation of Naïve Bayes in imbalanced classification tasks: its reliance on priors and likelihoods causes it to favor the majority class, especially when the minority class exhibits substantial feature overlap. Even with SMOTE applied, the posterior probability for fraud remained too low to cross the classification threshold. This suggests Naïve Bayes is unsuitable for real-world fraud detection, where missing fraud cases can have severe financial consequences. Although performance decreased on the pristine test set—an expected outcome when transitioning from balanced training data to real-world data—the final model met key industry benchmarks, demonstrating practical value for financial risk management.

Nevertheless, certain limitations emerged. While the Decision Tree met the minimum benchmarks in AUC and Recall, its Precision (0.2955) and F1-Score (0.4250) were notably lower than in validation. Future work could explore reducing the SMOTE ratio (e.g., to 3%) and incorporating dimensionality-reduction techniques—such as PCA—to enhance robustness and generalization. Additionally, more advanced ensemble models—such as Random Forest—could be introduced to benchmark against the Decision Tree, potentially offering improved performance and greater resilience to overfitting.

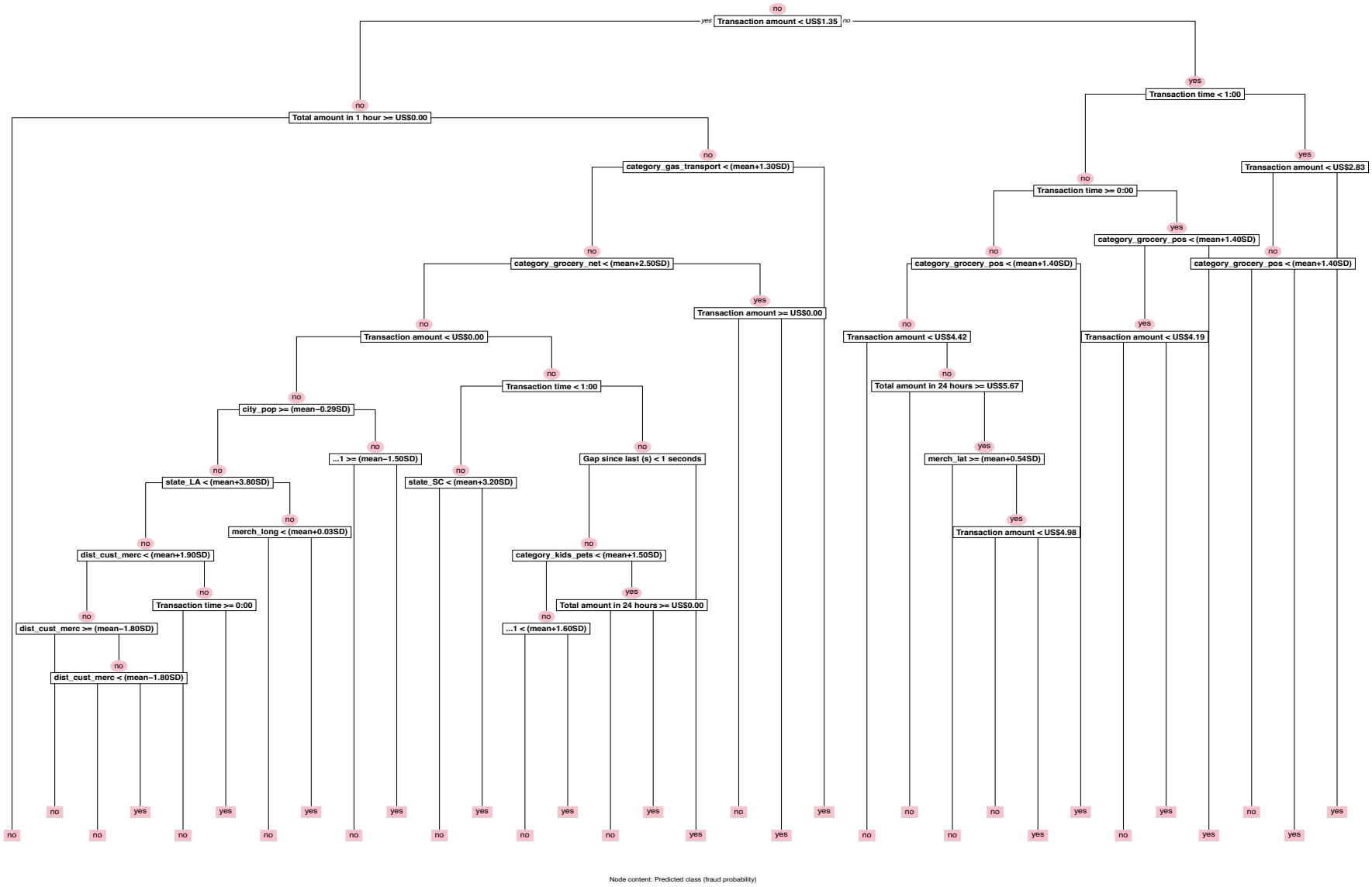
End

Appendix 1

(Figure A.1)

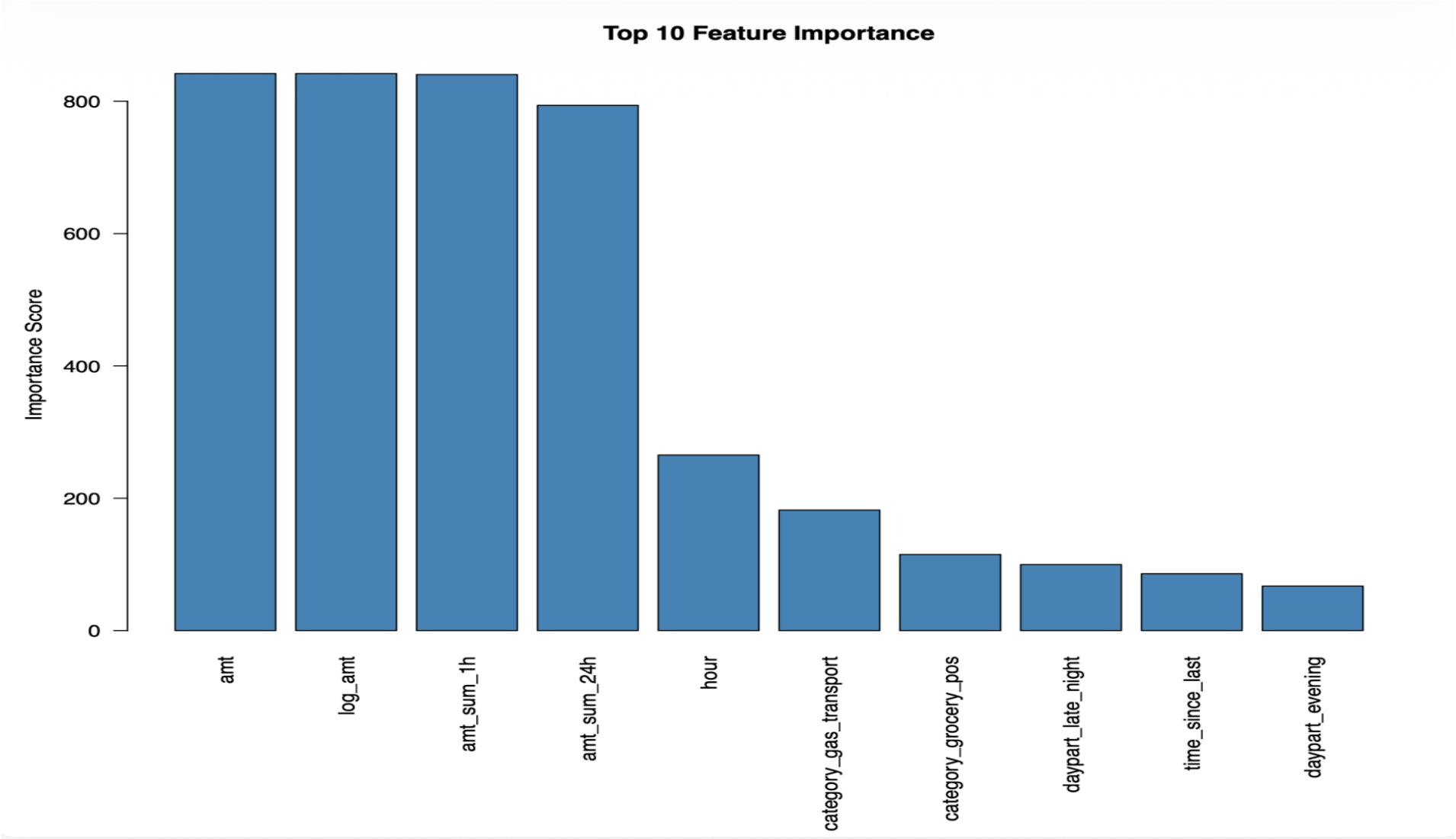


(Figure A.2)

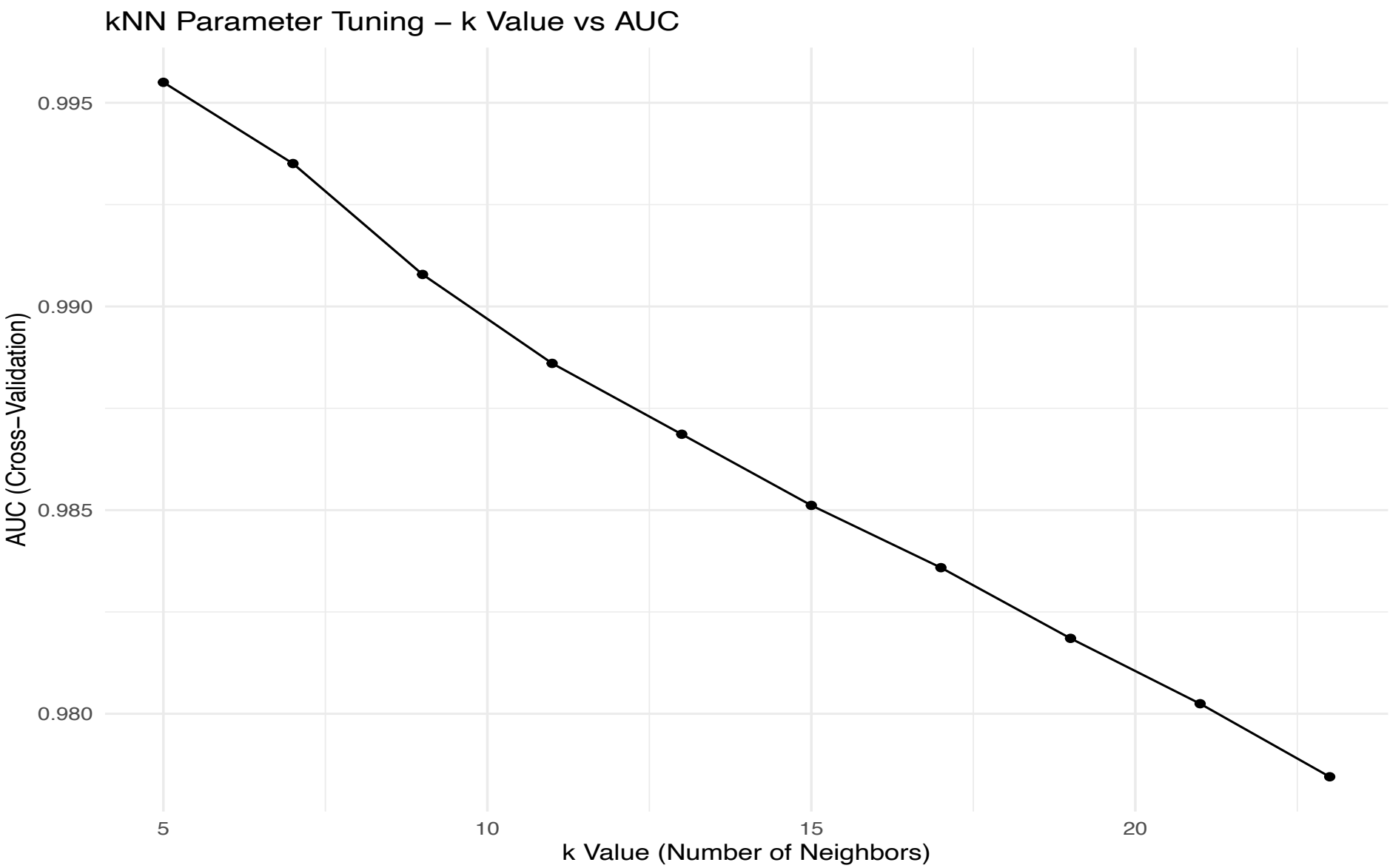


Node content: Predicted class (fraud probability)

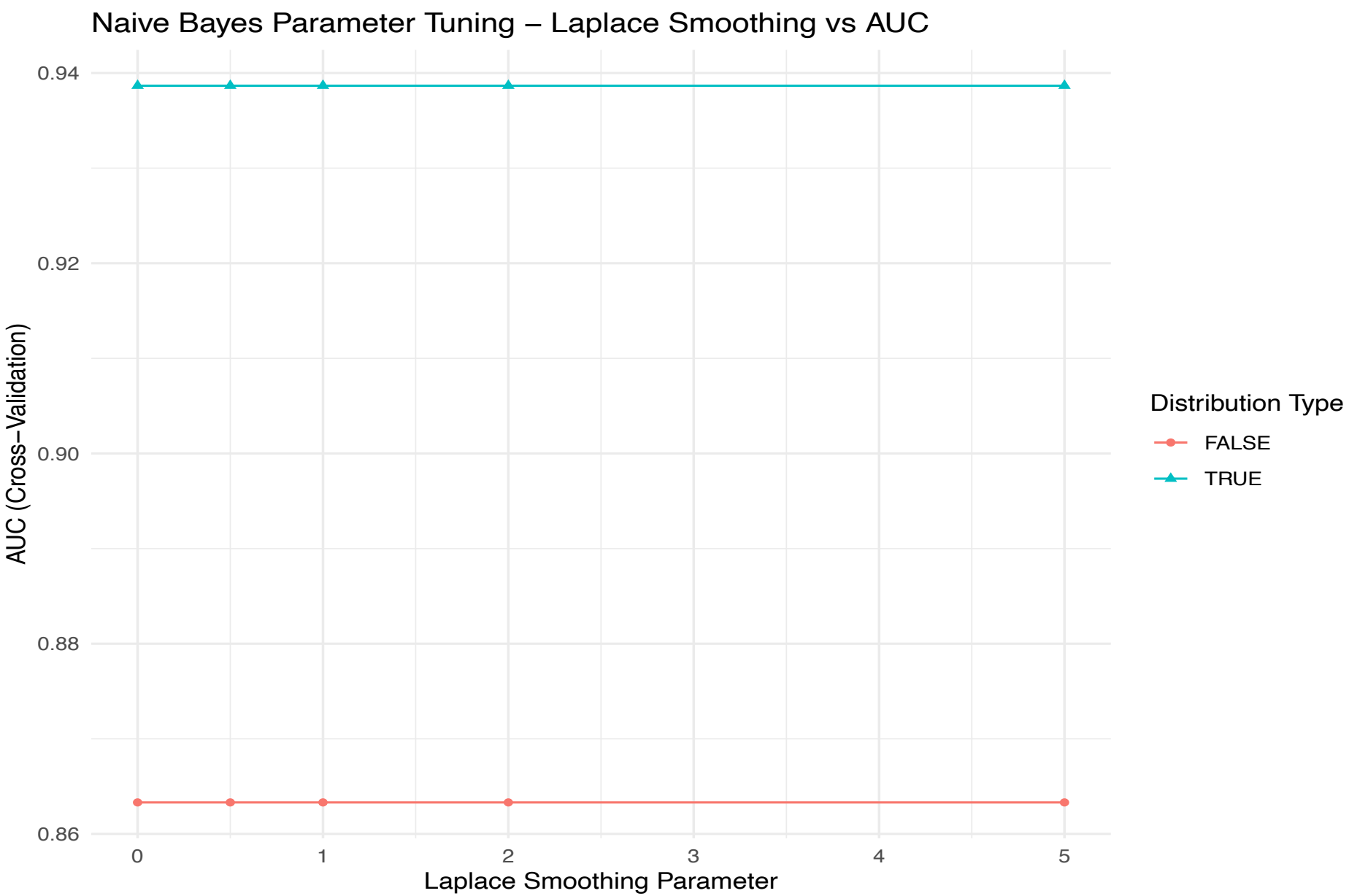
(Figure A.3)


Top 10 Feature Importance
Importance Score
800
600
400
200
0
amt
log_amt
amt_sum_1h
amt_sum_24h
hour
category_gas_transport
category_grocery_pos
daypart_late_night
time_since_last
daypart_evening

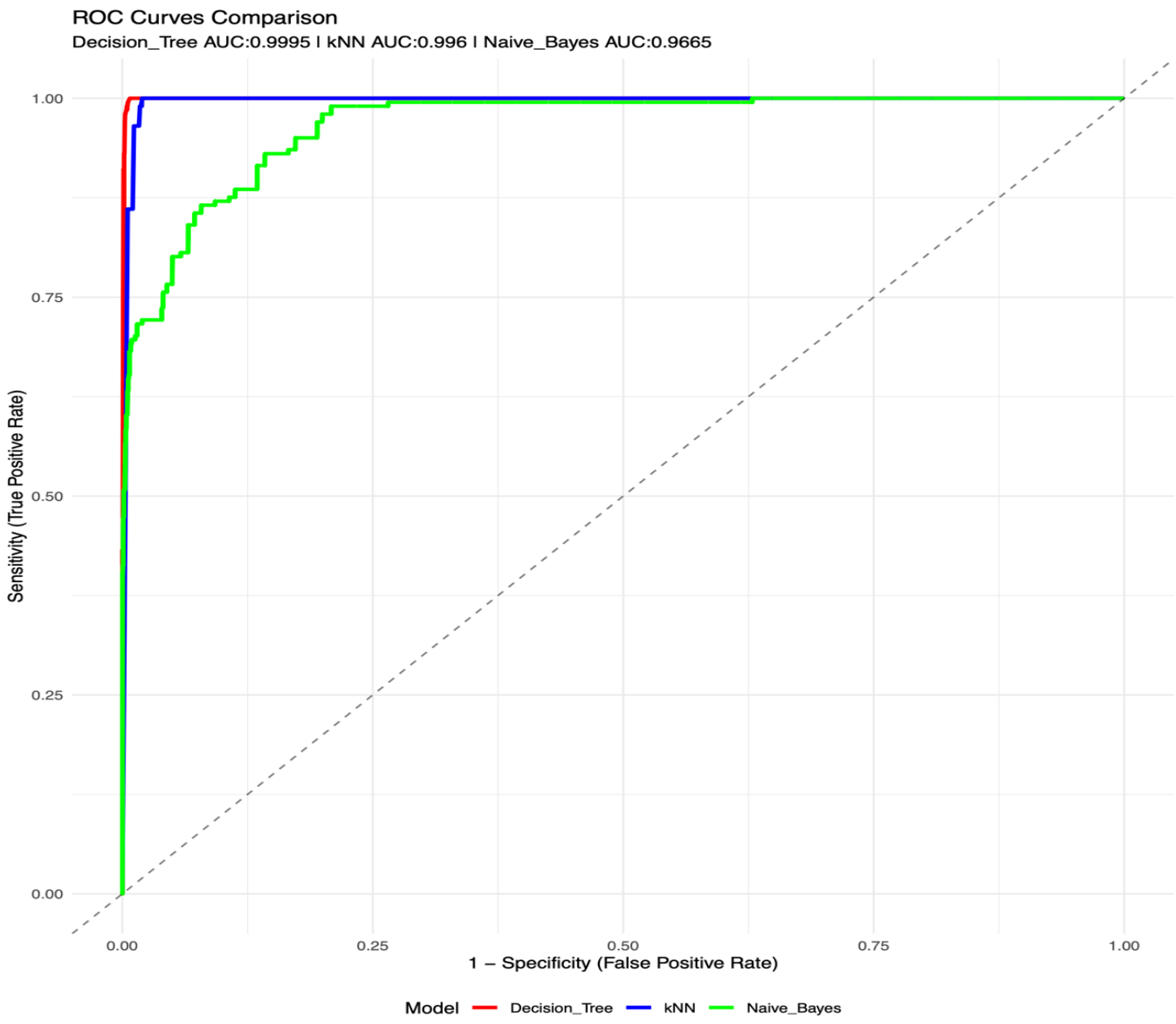
(Figure A.4)


kNN Parameter Tuning – k Value vs AUC
AUC (Cross–Validation)
0.995
0.990
0.985
0.980
5
10
15
20
k Value (Number of Neighbors)

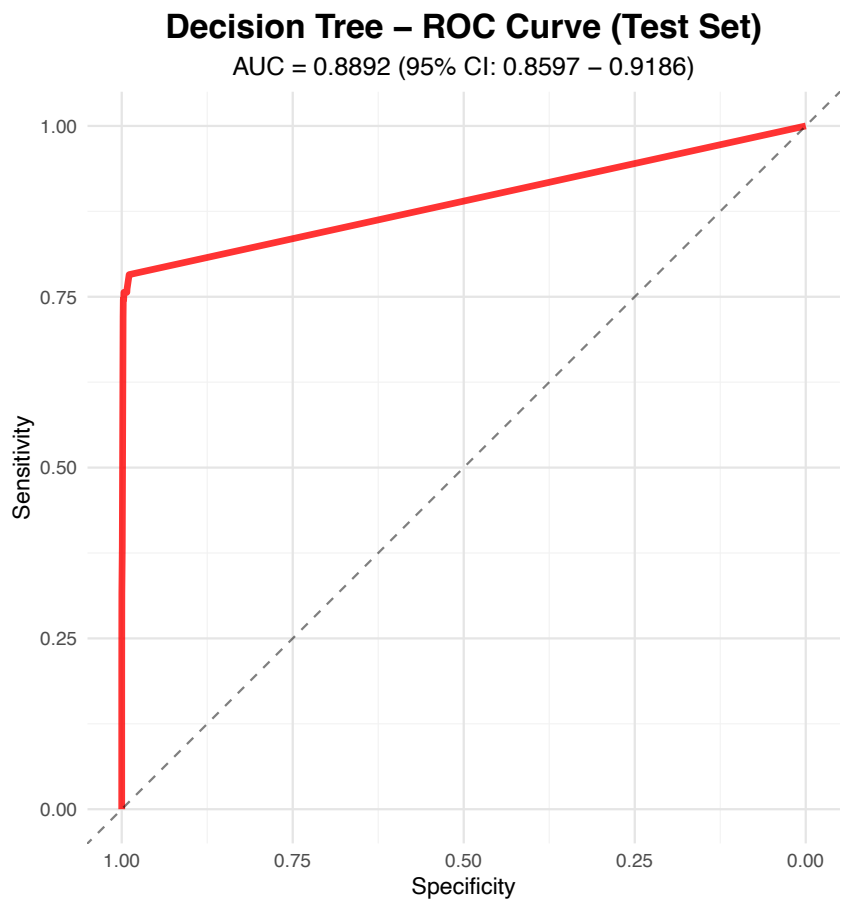
(Figure A.5)

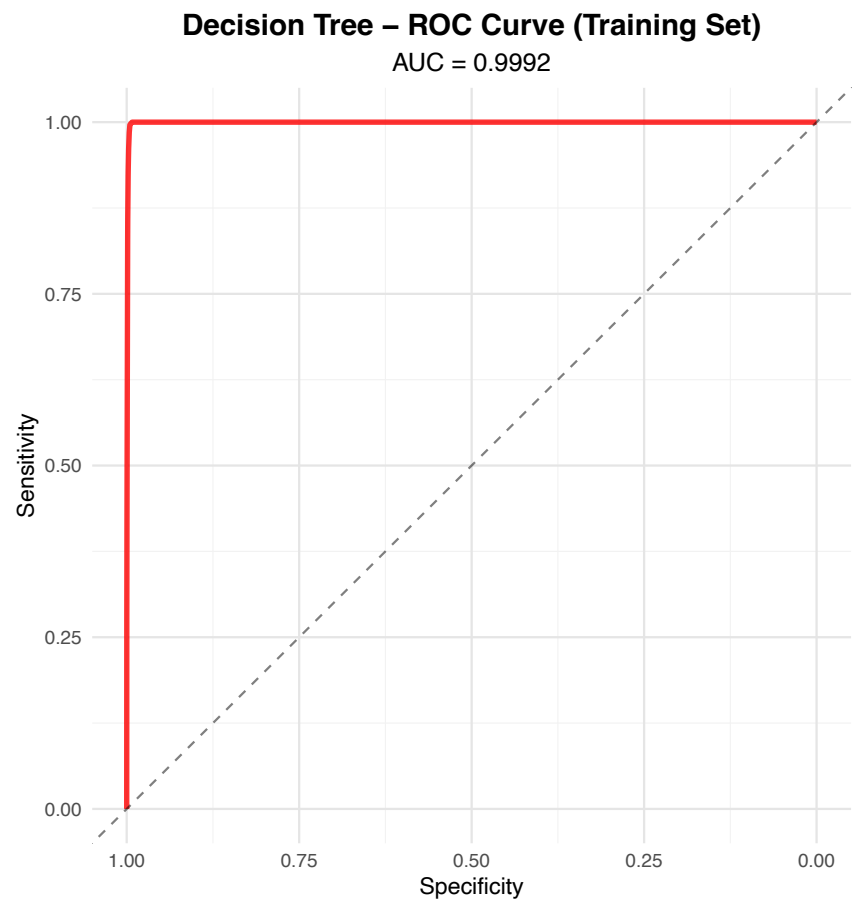

Naive Bayes Parameter Tuning – Laplace Smoothing vs AUC
0.94
0.92
0.90
0.88
0.86
AUC (Cross–Validation)
0
1
2
3
4
5
Laplace Smoothing Parameter
Distribution Type
FALSE
TRUE

(Figure A.6)


ROC Curves Comparison
Decision_Tree AUC:0.9995 I kNN AUC:0.996 I Naive_Bayes AUC:0.9665
Sensitivity (True Positive Rate)
1.00
0.75
0.50
0.25
0.00
0.00
0.25
0.50
0.75
1.00
1 – Specificity (False Positive Rate)
Model
Decision_Tree
kNN
Naive_Bayes

(Figure A.7)


Decision Tree – ROC Curve (Test Set)
AUC = 0.8892 (95% CI: 0.8597 – 0.9186)
Sensitivity
Specificity
1.00
0.75
0.50
0.25
0.00


Decision Tree – ROC Curve (Training Set)
AUC = 0.9992
Sensitivity
Specificity
1.00
0.75
0.50
0.25
0.00

References list (APA 7th Edition)

- Bahnsen, A. C., Aouada, D., Stojanovic, A., & Ottersten, B. (2016). Feature engineering strategies for credit card fraud detection. Expert Systems with Applications, 51, 134–142. https://doi.org/10.1016/j.eswa.2015.12.030
- Bahnsen, A. M., Aouada, D., Stojanovic, A., & Ottersten, B. (2016). Feature engineering for credit card fraud detection. Expert Systems with Applications, 51, 134–142. https://doi.org/10.1016/j.eswa.2015.12.007
- Blagus, R., & Lusa, L. (2013). SMOTE for high-dimensional class-imbalanced data. BMC Bioinformatics, 14, 106. https://doi.org/10.1186/1471-2105-14-106
- Bolton, R. J., & Hand, D. J. (2002). Statistical fraud detection: A review. Statistical Science, 17(3), 235–255. https://doi.org/10.1214/ss/1042727940
- Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). Classification and regression trees. Wadsworth.
- Carcillo, A., Dal Pozzolo, A., Le Borgne, Y.-A., Caelen, O., & Bontempi, G. (2019). Combining unsupervised and supervised learning in credit card fraud detection. Information Sciences, 517, 199–215.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357. https://doi.org/10.1613/jair.953
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21–27. https://doi.org/10.1109/TIT.1967.1057541
- Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2014). Learned lessons in credit card fraud detection from a practitioner perspective. Expert Systems with Applications, 41(10), 4915–4928. https://doi.org/10.1016/j.eswa.2014.02.026
- Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 374(2065), 20150202. https://doi.org/10.1098/rsta.2015.0202
- Kartik. (2020). Credit Card Fraud Detection Dataset [Data set]. Kaggle. https://www.kaggle.com/datasets/kartik2112/fraud-detection
- Kuhn, M., & Johnson, K. (2013). Applied predictive modeling. Springer. https://doi.org/10.1007/978-1-4614-6849-3
- Kuhn, M., & Silge, J. (2022). Tidy modeling with R. O'Reilly Media. https://www.tmwr.org
- López, V., Fernández, A., García, S., Palade, V., & Herrera, F. (2013). An insight into classification with imbalanced data: Empirical results and current trends. Information Sciences, 250, 113–141. https://doi.org/10.1016/j.ins.2013.07.007
- Saito, T., & Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLOS ONE, 10(3), e0118432. https://doi.org/10.1371/journal.pone.0118432
- Sinnott, R. W. (1984). Virtues of the haversine. Sky and Telescope, 68(2), 159.

- Torgo, L. (2010). Data mining with R: Learning with case studies. Chapman & Hall/CRC.